

martinfowler.com

Emerging Patterns in Building GenAI Products

Bharani Subramaniam

36–45 minutes

The transition of Generative AI powered products from proof-of-concept to production has proven to be a significant challenge for software engineers everywhere. We believe that a lot of these difficulties come from folks thinking that these products are merely extensions to traditional transactional or analytical systems. In our engagements with this technology we've found that they introduce a whole new range of problems, including hallucination, unbounded data access and non-determinism.

We've observed our teams follow some regular patterns to deal with these problems. This article is our effort to capture these. This is early days for these systems, we are learning new things with every phase of the moon, and new tools [flood our radar](#). As with any pattern, none of these are gold standards that should be used in all circumstances. The notes on when to use it are often more important than the description of how it works.

In this article we describe the patterns briefly, interspersed with narrative text to better explain context and interconnections. We've identified the pattern sections with the “❖” dingbat. Any section that describes a pattern has the title surrounded by a single ❖. The pattern description

ends with “❖ ❖ ❖”

These patterns are our attempt to understand what *we have seen* in our engagements. There's a lot of research and tutorial writing on these systems out there, and some decent books are beginning to appear to act as general education on these systems and how to use them. This article is not an attempt to be such a general education, rather it's trying to organize the experience that our colleagues have had using these systems in the field. As such there will be gaps where we haven't tried some things, or we've tried them, but not enough to discern any useful pattern. As we work further we intend to revise and expand this material, as we extend this article we'll send updates to [our usual feeds](#).

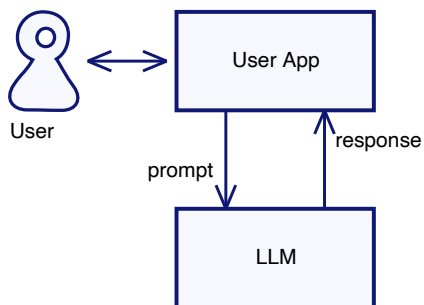
Patterns in this Article

Direct Prompting	Send prompts directly from the user to a Foundation LLM
Embeddings	Transform large data blocks into numeric vectors so that embeddings near each other represent related concepts
Evals	Evaluate the responses of an LLM in the context of a specific task
Hybrid Retriever	Combine searches using embeddings with other search techniques
Query Rewriting	Use an LLM to create several alternative formulations of a query and search with all the alternatives

Reranker	Rank a set of retrieved document fragments according to their usefulness and send the best of them to the LLM.
Retrieval Augmented Generation (RAG)	Retrieve relevant document fragments and include these when prompting the LLM

Direct Prompting

Send prompts directly from the user to a Foundation LLM



The most basic approach to using an LLM is to connect an off-the-shelf LLM directly to a user, allowing the user to type prompts to the LLM and receive responses without any intermediate steps. This is the kind of experience that LLM vendors may offer directly.

When to use it

While this is useful in many contexts, and its usage triggered the wide excitement about using LLMs, it has some significant shortcomings.

The first problem is that the LLM is constrained by the data it was trained on. This means that the LLM will not know anything that has happened since it was trained. It also means that the LLM will be unaware of specific information that's outside of its training set. Indeed even if it's within the training set, it's still unaware of the context that's operating in,

which should make it prioritize some parts of its knowledge base that's more relevant to this context.

As well as knowledge base limitations, there are also concerns about how the LLM will behave, particularly when faced with malicious prompts. Can it be tricked to divulging confidential information, or to giving misleading replies that can cause problems for the organization hosting the LLM. LLMs have a habit of showing confidence even when their knowledge is weak, and freely making up plausible but nonsensical answers. While this can be amusing, it becomes a serious liability if the LLM is acting as a spoke-bot for an organization.

[Direct Prompting](#) is a powerful tool, but one that often cannot be used alone. We've found that for our clients to use LLMs in practice, they need additional measures to deal with the limitations and problems that [Direct Prompting](#) alone brings with it.

The first step we need to take is to figure out how good the results of an LLM really are. In our regular software development work we've learned the value of putting a strong emphasis on testing, checking that our systems reliably behave the way we intend them to. When evolving our practices to work with Gen AI, we've found it's crucial to establish a systematic approach for evaluating the effectiveness of a model's responses. This ensures that any enhancements—whether structural or contextual—are truly improving the model's performance and aligning with the intended goals. In the world of gen-ai, this leads to...

Evals

Evaluate the responses of an LLM in the context of a specific task

Whenever we build a software system, we need to ensure that it behaves in a way that matches our intentions. With traditional systems,

we do this primarily through testing. We provided a thoughtfully selected sample of input, and verified that the system responds in the way we expect.

With LLM-based systems, we encounter a system that no longer behaves deterministically. Such a system will provide different outputs to the same inputs on repeated requests. This doesn't mean we cannot examine its behavior to ensure it matches our intentions, but it does mean we have to think about it differently.

The Gen-AI examines behavior through “evaluations”, usually shortened to “evals”. Although it is possible to evaluate the model on individual output, it is more common to assess its behavior across a range of scenarios. This approach ensures that all anticipated situations are addressed and the model's outputs meet the desired standards.

Scoring and Judging

Necessary arguments are fed through a scorer, which is a component or function that assigns numerical scores to generated outputs, reflecting evaluation metrics like relevance, coherence, factuality, or semantic similarity between the model's output and the expected answer.

Model Input

Model Output

Expected Output

Retrieval context from RAG

Metrics to evaluate
(accuracy, relevance...)

Performance Score

Ranking of Results

Additional Feedback

Different evaluation techniques exist based on who computes the score, raising the question: who, ultimately, will act as the judge?

- **Self evaluation:** Self-evaluation lets LLMs self-assess and enhance their own responses. Although some LLMs can do this better than others, there is a critical risk with this approach. If the model's internal self-assessment process is flawed, it may produce outputs that appear more confident or refined than they truly are, leading to reinforcement of errors or biases in subsequent evaluations. While self-evaluation exists as a technique, we strongly recommend exploring other strategies.
- **LLM as a judge:** The output of the LLM is evaluated by scoring it with another model, which can either be a more capable LLM or a specialized Small Language Model (SLM). While this approach involves evaluating with an LLM, using a different LLM helps address some of the issues of self-evaluation. Since the likelihood of both models sharing the same errors or biases is low, this technique has become a popular choice for automating the evaluation process.
- **Human evaluation:** Vibe checking is a technique to evaluate if the LLM responses match the desired tone, style, and intent. It is an informal way to assess if the model "gets it" and responds in a way that feels right for the situation. In this technique, humans manually write prompts and evaluate the responses. While challenging to scale, it's the most effective method for checking qualitative elements that automated methods typically miss.

In our experience, combining LLM as a judge with human evaluation works better for gaining an overall sense of how LLM is performing on

key aspects of your Gen AI product. This combination enhances the evaluation process by leveraging both automated judgment and human insight, ensuring a more comprehensive understanding of LLM performance.

Example

Here is how we can use [DeepEval](#) to test the relevancy of LLM responses from our nutrition app

```
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric

def test_answer_relevancy():
    answer_relevancy_metric = AnswerRelevancyMetric(threshold=0.5)
    test_case = LLMTestCase(
        input="What is the recommended daily protein intake for adults?",
        actual_output="The recommended daily protein intake for adults is 0.8 grams per kilogram of body weight.",
        retrieval_context=[
            "Protein is an essential macronutrient that plays crucial roles in building and repairing tissues. Good sources include lean meats, fish, eggs, and legumes. The recommended daily allowance (RDA) for protein is 0.8 grams per kilogram of body weight for adults. Athletes and active individuals may need more, ranging from 1.2 to 2.0 grams per kilogram of body weight."
        ]
    )
    assert_test(test_case, [answer_relevancy_metric])
```

In this test, we evaluate the LLM response by embedding it directly and measuring its relevance score. We can also consider adding integration tests that generate live LLM outputs and measure it across a number of [pre-defined metrics](#).

Running the Evals

As with testing, we run evals as part of the build pipeline for a Gen-AI system. Unlike tests, they aren't simple binary pass/fail results, instead we have to set thresholds, together with checks to ensure performance doesn't decline. In many ways we treat evals similarly to how we work with performance testing.

Our use of evals isn't confined to pre-deployment. A live gen-AI system may change its performance while in production. So we need to carry out regular evaluations of the deployed production system, again looking for any decline in our scores.

Evaluations can be used against the whole system, and against any components that have an LLM. Guardrails and [Query Rewriting](#) contain logically distinct LLMs, and can be evaluated individually, as well as part of the total request flow.

Evals and Benchmarking

Benchmarking is the process of establishing a baseline for comparing the output of LLMs for a well defined set of tasks. In benchmarking, the goal is to minimize variability as much as possible. This is achieved by using standardized datasets, clearly defined tasks, and established metrics to consistently track model performance over time. So when a new version of the model is released you can compare different metrics and take an informed decision to upgrade or stay with the current

version.

LLM creators typically handle benchmarking to assess overall model quality. As a Gen AI product owner, we can use these benchmarks to gauge how well the model performs in general. However, to determine if it's suitable for our specific problem, we need to perform targeted evaluations.

Unlike generic benchmarking, evals are used to measure the output of LLM for our specific task. There is no industry established dataset for evals, we have to create one that best suits our use case.

When to use it

Assessing the accuracy and value of any software system is important, we don't want users to make bad decisions based on our software's behavior. The difficult part of using evals lies in fact that it is still early days in our understanding of what mechanisms are best for scoring and judging. Despite this, we see evals as crucial to using LLM-based systems outside of situations where we can be comfortable that users treat the LLM-system with a healthy amount of skepticism.

[Evals](#) provide a vital mechanism to consider the broad behavior of a generative AI powered system. We now need to turn to looking at how to structure that behavior. Before we can go there, however, we need to understand an important foundation for generative, and other AI based, systems: how they work with the vast amounts of data that they are trained on, and manipulate to determine their output.

Embeddings

Transform large data blocks into numeric vectors so that embeddings near each other represent related concepts



Imagine you're creating a nutrition app. Users can snap photos of their meals and receive personalized tips and alternatives based on their lifestyle. Even a simple photo of an apple taken with your phone contains a vast amount of data. At a resolution of 1280 by 960, a single image has around 3.6 million pixel values (1280 x 960 x 3 for RGB). Analyzing patterns in such a large dimensional dataset is impractical even for smartest models.

An embedding is lossy compression of that data into a large numeric vector, by “large” we mean a vector with several hundred elements . This transformation is done in such a way that similar images transform into vectors that are close to each other in this hyper-dimensional space.

Example Image Embedding

Deep learning models create more effective image embeddings than hand-crafted approaches. Therefore, we'll use a CLIP (Contrastive Language-Image Pre-Training) model, specifically [clip-ViT-L-14](#), to generate them.

```
# python
from sentence_transformers import SentenceTransformer, util
from PIL import Image
import numpy as np

model = SentenceTransformer('clip-ViT-L-14')
apple_embeddings = model.encode(Image.open('images/Apple/
```

```
Apple_1.jpeg'))
```

```
print(len(apple_embeddings)) # Dimension of embeddings 768
```

```
print(np.round(apple_embeddings, decimals=2))
```

If we run this, it will print out how long the embedding vector is, followed by the vector itself

```
768
```

```
[0.3  0.25 0.83 0.33 -0.05 0.39 -0.67 0.13 0.39 0.5 # and so on...
```

768 numbers are a lot less data to work with than the original 3.6 million.

Now that we have compact representation, let's also test the hypothesis that similar images should be located close to each other in vector space. There are several approaches to determine the distance between two embeddings, including cosine similarity and Euclidean distance.

For our nutrition app we will use cosine similarity. The cosine value ranges from -1 to 1:

cosine value	vectors	result
1	perfectly aligned	images are highly similar
-1	perfectly anti-aligned	images are highly dissimilar
0	orthogonal	images are unrelated

Given two embeddings, we can compute cosine similarity score as:

```
def cosine_similarity(embedding1, embedding2):  
    embedding1 = embedding1 / np.linalg.norm(embedding1)  
    embedding2 = embedding2 / np.linalg.norm(embedding2)  
    cosine_sim = np.dot(embedding1, embedding2)  
    return cosine_sim
```

Let's now use the following images to test our hypothesis with the following four images.



apple 1



apple 2



apple 3





burger

Here's the results of comparing apple 1 to the four iamges

image	cosine_similarity	remarks
apple 1	1.0	same picture, so perfect match
apple 2	0.9229323	similar, so close match
apple 3	0.8406111	close, but a bit further away
burger	0.58842075	quite far away

In reality there could be a number of variations - What if the apples are cut? What if you have them on a plate? What if you have green apples? What if you take a top view of the apple? The embedding model should encode meaningful relationships and represent them efficiently so that similar images are placed in close proximity.

It would be ideal if we can somehow visualize the embeddings and verify the clusters of similar images. Even though ML models can comfortably work with 100s of dimensions, to visualize them we may have to further reduce the dimensions ,using techniques like [T-SNE](#) or [UMAP](#) , so that we can plot embeddings in two or three dimensional space.

Here is a handy T-SNE method to do just that

```
from sklearn.manifold import TSNE
tsne = TSNE(random_state = 0, metric =
'cosine',perplexity=2,n_components = 3)
embeddings_3d = tsne.fit_transform(array_of_embeddings)
```

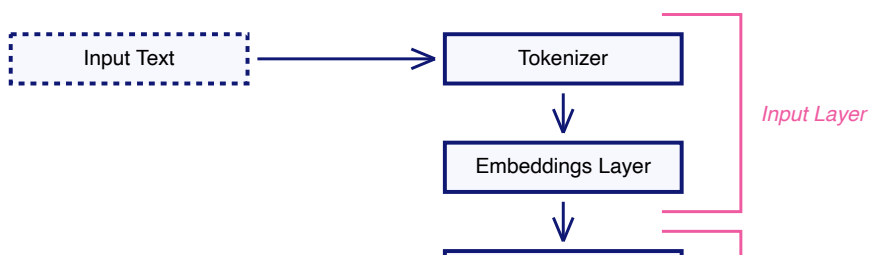
Now that we have a 3 dimensional array, we can visualize embeddings of images from Kaggle's [fruit classification dataset](#)

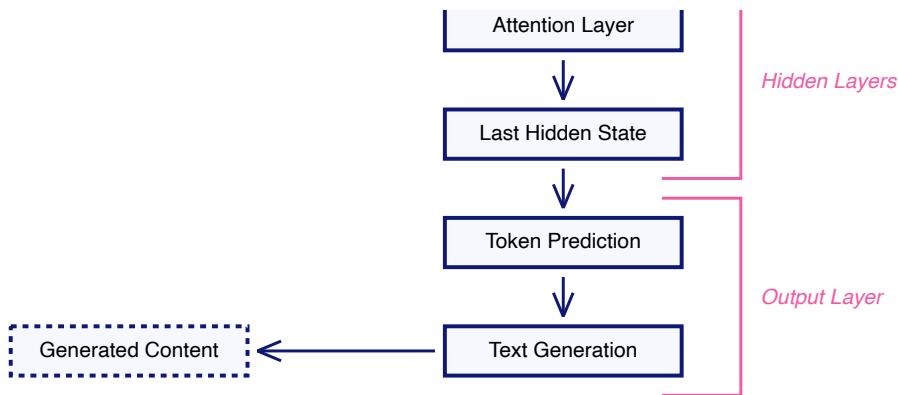
The embeddings model does a pretty good job of clustering embeddings of similar images close to each other.

So this is all very well for images, but how does this apply to documents? Essentially there isn't much to change, a chunk of text, or pages of text, images, and tables - these are just data. An embeddings model can take several pages of text, and convert them into a vector space for comparison. Ideally it doesn't just take raw words, instead it understands the context of the prose. After all “Mary had a little lamb” means one thing to a teller of nursery rhymes, and something entirely different to a restaurateur. Models like [text-embedding-3-large](#) and [all-MiniLM-L6-v2](#) can capture complex semantic relationships between words and phrases.

Embeddings in LLM

LLMs are specialized neural networks known as [Transformers](#). While their internal structure is intricate, they can be conceptually divided into an input layer, multiple hidden layers, and an output layer.





A significant part of the input layer consists of embeddings for the vocabulary of the LLM. These are called internal, parametric, or static embeddings of the LLM.

Back to our nutrition app, when you snap a picture of your meal and ask the model

“Is this meal healthy?”



The LLM does the following logical steps to generate the response

- At the input layer, the tokenizer converts the input prompt texts and

images to embeddings.

- Then these embeddings are passed to the LLM's internal hidden layers, also called attention layers, that extracts relevant features present in the input. Assuming our model is trained on nutritional data, different attention layers analyze the input from health and nutritional aspects
- Finally, the output from the last hidden state, which is the last attention layer, is used to predict the output.

When to use it

Embeddings capture the meaning of data in a way that enables semantic similarity comparisons between items, such as text or images. Unlike surface-level matching of keywords or patterns, embeddings encode deeper relationships and contextual meaning.

As such, generating embeddings involves running specialized AI models, which are typically smaller and more efficient than large language models. Once created, embeddings can be used for similarity comparisons efficiently, often relying on simple vector operations like cosine similarity

However, embeddings are not ideal for structured or relational data, where exact matching or traditional database queries are more appropriate. Tasks such as finding exact matches, performing numerical comparisons, or querying relationships are better suited for SQL and traditional databases than embeddings and vector stores.

We started this discussion by outlining the limitations of [Direct Prompting](#). [Evals](#) give us a way to assess the overall capability of our system, and [Embeddings](#) provides a way to index large quantities of unstructured data. LLMs are trained, or as the community says “pre-

trained” on a corpus of this data. For general cases, this is fine, but if we want a model to make use of more specific or recent information, we need the LLM to be aware of data outside this pre-training set.

One way to adapt a model to a specific task or domain is to carry out extra training, known as Fine Tuning. The trouble with this is that it's very expensive to do, and thus usually not the best approach. (We'll explore when it can be the right thing later.) For most situations, we've found the best path to take is that of RAG.

Retrieval Augmented Generation (RAG)

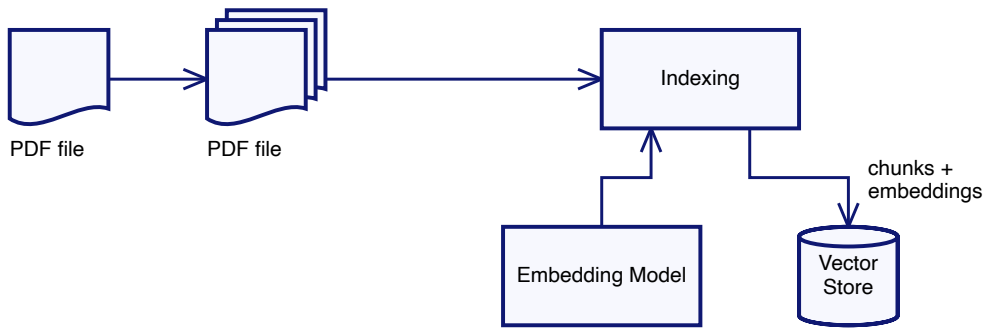
Retrieve relevant document fragments and include these when prompting the LLM

A common metaphor for an LLM is a junior researcher. Someone who is articulate, well-read in general, but not well-informed on the details of the topic - and woefully over-confident, preferring to make up a plausible answer rather than admit ignorance. With RAG, we are asking this researcher a question, and also handing them a dossier of the most relevant documents, telling them to read those documents before coming up with an answer.

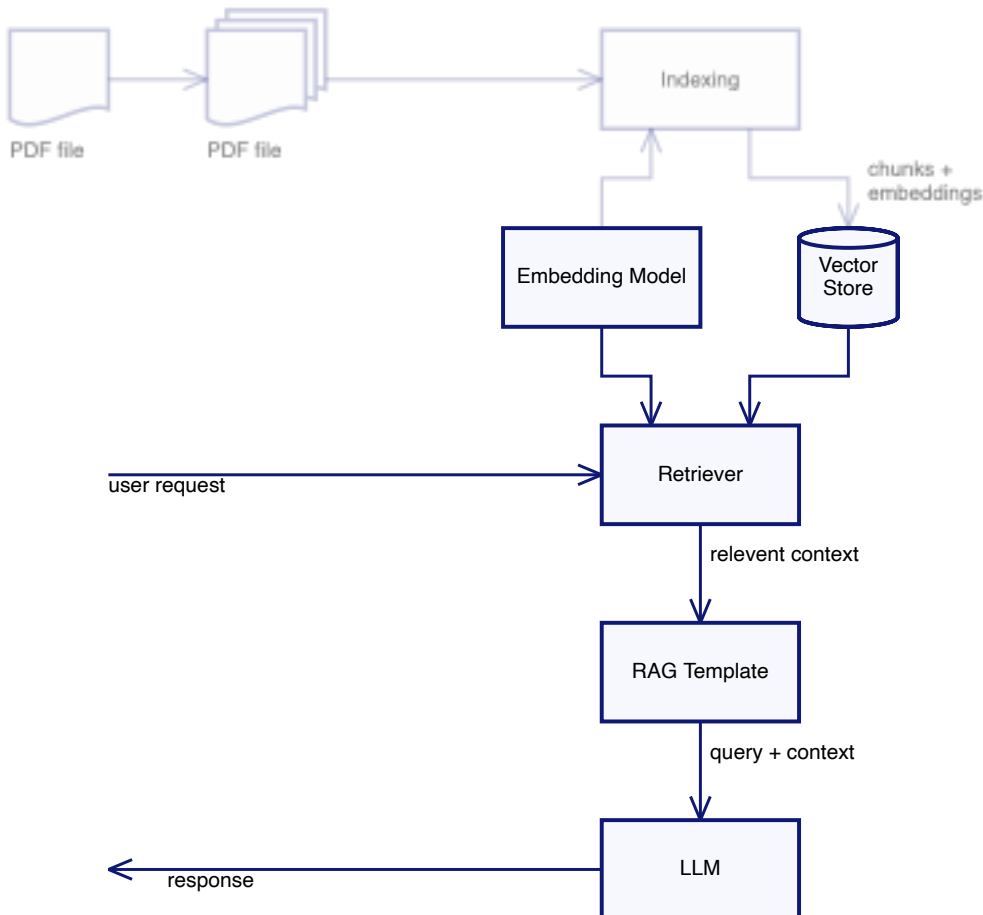
We've found RAGs to be an effective approach for using an LLM with specialized knowledge. But they lead to classic Information Retrieval (IR) problems - how do we find the right documents to give to our eager researcher?

The common approach is to build an index to the documents using embeddings, then use this index to search the documents.

The first part of this is to build the index. We do this by dividing the documents into chunks, creating embeddings for the chunks, and saving the chunks and their embeddings into a vector database.



We then handle user requests by using the embedding model to create an embedding for the query. We use that embedding with a ANN similarity search on the vector store to retrieve matching fragments. Next we use the RAG prompt template to combine the results with the original query, and send the complete input to the LLM.



RAG Template

Once we have document fragments from the retriever, we then combine

the users prompt with these fragments using a prompt template. We also add instructions to explicitly direct the LLM to use this context and to recognize when it lacks sufficient data.

Such a prompt template may look like this

User prompt: {{user_query}}

Relevant context: {{retrieved_text}}

Instructions:

- 1. Provide a comprehensive, accurate, and coherent response to the user query, using the provided context.
- 2. If the retrieved context is sufficient, focus on delivering precise and relevant information.
- 3. If the retrieved context is insufficient, acknowledge the gap and suggest potential sources or steps for obtaining more information.
- 4. Avoid introducing unsupported information or speculation.

When to use it

By supplying an LLM with relevant information in its query, RAG surmounts the limitation that an LLM can only respond based on its training data. It combines the strengths of information retrieval and generative models

RAG is particularly effective for processing rapidly changing data, such as news articles, stock prices, or medical research. It can quickly retrieve the latest information and integrate it into the LLM's response, providing a more accurate and contextually relevant answer.

RAG enhances the factuality of LLM responses by accessing and incorporating relevant information from a knowledge base, minimizing

the risk of hallucinations or fabricated content. It is easy for the LLM to include references to the documents it was given as part of its context, allowing the user to verify its analysis.

The context provided by the retrieved documents can mitigate biases in the training data. Additionally, RAG can leverage in-context learning (ICL) by embedding task specific examples or patterns in the retrieved content, enabling the model to dynamically adapt to new tasks or queries.

An alternative approach for extending the knowledge base of an LLM is Fine Tuning, which we'll discuss later. Fine-tuning requires substantially greater resources, and thus most of the time we've found RAG to be more effective.

RAG in Practice

Our description above is what we consider a basic RAG, much along the lines that was described in the original paper.¹ We've used RAG in a number of engagements and found it's an effective way to use LLMs to interact with a large and unruly dataset. However, we've also found the need to make many enhancements to the basic idea to make this work with serious problem.

1: The term “RAG” was originally coined in a [paper](#) by a group of researchers at Meta AI. Like many academic papers, it isn't an easy read, but a subset of those authors also wrote a [blog post](#) that's more approachable.

One example we will highlight is some work we did building a query system for a multinational life sciences company. Researchers at this company often need to survey details of past studies on various compounds and species. These studies were made over two decades of

research, yielding 17,000 reports, each with thousands of pages containing both text and tabular data. We built a chatbot that allowed the researchers to query this trove of sporadically structured data.

Before this project, answering complex questions often involved manually sifting through numerous PDF documents. This could take a few days to weeks. Now, researchers can leverage multi-hop queries in our chatbot and find the information they need in just a few minutes. We have also incorporated visualizations where needed to ease exploration of the dataset used in the reports.

This was a successful use of RAG, but to take it from a proof-of-concept to a viable production application, we needed to overcome several serious limitations.

2: Interesting alternative to chunk embeddings is [ColBERT](#). Instead of encoding entire paragraphs as single embeddings, ColBERT represents each passage as a matrix of token-level embeddings.

Limitation		Mitigating Pattern
Inefficient retrieval	When you're just starting with retrieval systems, it's a shock to realize that relying solely on document chunk embeddings in a vector store won't lead to efficient retrieval. The common assumption is that chunk embeddings alone will work, but in reality it is useful but not very effective on its own. When we create a single embedding vector for a document chunk, we compress multiple paragraphs into one dense vector. While dense embeddings	Hybrid Retriever

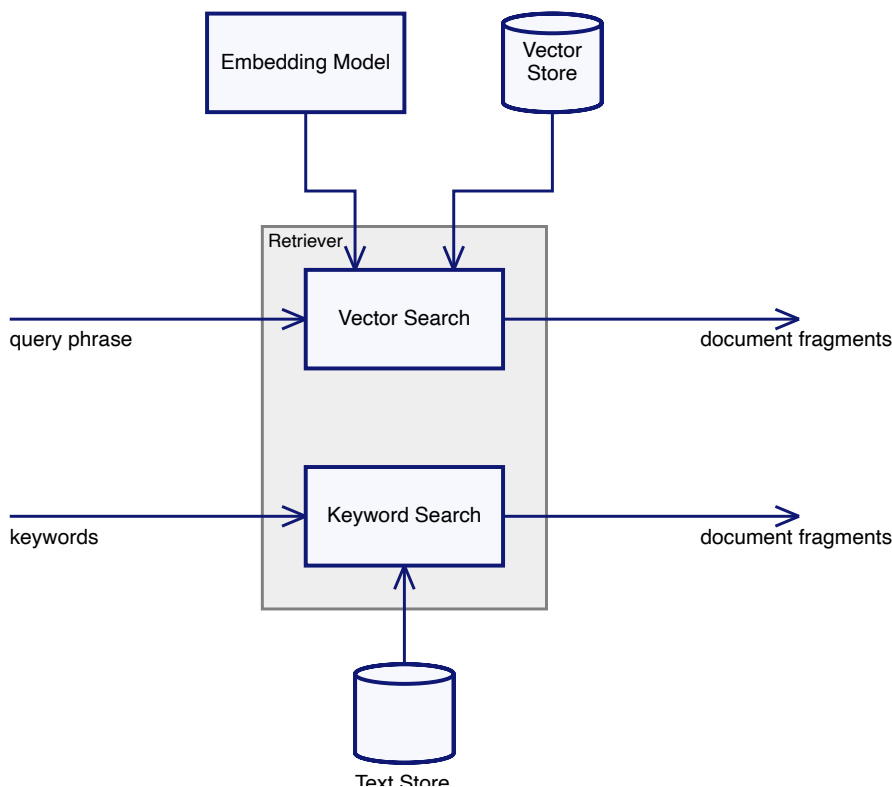
Limitation		Mitigating Pattern
	are good at finding similar paragraphs, they inevitably lose some semantic detail. No amount of fine-tuning can completely bridge this gap. ²	
Minimalistic user query	Not all users are able to clearly articulate their intent in a well-formed natural language query. Often, queries are short and ambiguous, lacking the specificity needed to retrieve the most relevant documents. Without clear keywords or context, the retriever may pull in a broad range of information, including irrelevant content, which leads to less accurate and more generalized results.	Query Rewriting
Context bloat	The Lost in the Middle paper reveals that LLMs currently struggle to effectively leverage information within lengthy input contexts. Performance is generally strongest when relevant details are positioned at the beginning or end of the context. However, it drops considerably when models must retrieve critical information from the middle of long inputs. This limitation persists even in models specifically designed for large context.	Reranker

Limitation		Mitigating Pattern
Gullibility	We characterized LLMs earlier as like a junior researcher: articulate, well-read, but not well-informed on specifics. There's another adjective we should apply: gullible. Our AI researchers are easily convinced to say things better left silent, revealing secrets, or making things up in order to appear more knowledgeable than they are.	Guardrails

As the above indicates, each limitation is a problem that spurs a pattern to address it

Hybrid Retriever

Combine searches using embeddings with other search techniques



Text Search

While vector operations on embeddings of text is a powerful and sophisticated technique, there's a lot to be said for simple keyword searches. Techniques like [TF/IDF](#) and [BM25](#), are mature ways to efficiently match exact terms. We can use them to make a faster and less compute-intensive search across the large document set, finding candidates that a vector search alone wouldn't surface. Combining these candidates with the result of the vector search, yields a better set of candidates. The downside is that it can lead to an overly large set of documents for the LLM, but this can be dealt with by using a [reranker](#).

When we use a hybrid retriever, we need to supplement the indexing process to prepare our data for the vector searches. We experimented with different chunk sizes and settled on 1000 characters with 100 characters of overlap. This allowed us to focus the LLM's attention onto the most relevant bits of context. While model context lengths are increasing, current research indicates that accuracy diminishes with larger prompts. For embeddings we used OpenAI's [text-embedding-3-large](#) model to process the chunks, generating embeddings that we stored in AWS OpenSearch.

Let us consider a simple JSON document like

```
{
  "Title": "title of the research",
  "Description": "chunks of the document approx 1000 bytes"
}
```

For normal text based keyword search, it is enough to simply insert this document and create a "text" index on top of either title or description. However, for vector search on description we have to explicitly add an additional field to store its corresponding embedding.

```
{  
  "Title": "title of the research",  
  "Description": "chunks of the document approx 1000 bytes",  
  "Description_Vec": [1.23, 1.924, ...] // embeddings vector created via  
embedding model  
}
```

With this setup, we can create both text based search on title and description as well as vector search on `description_vec` fields.

When to use it

Embeddings are a powerful way to find chunks of unstructured data. They naturally fit with using LLMs because they play an important role within the LLM themselves. But often there are characteristics of the data that allow alternative search approaches, which can be used in addition.

Indeed sometimes we don't need to use vector searches at all in the retriever. In our work [using AI to help understand legacy code](#), we used the Neo4J graph database to hold a representation of the Abstract Syntax Tree of the codebase, and annotated the nodes of that tree with data gleaned from documentation and other sources. In our experiments, we observed that representing dependencies of modules, function call and caller relationships as a graph is more straightforward and effective than using embeddings.

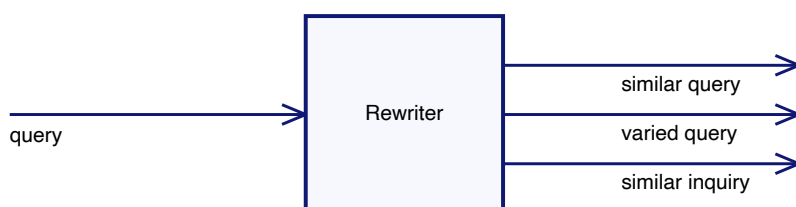
That said, embeddings still played a role here, as we used them with an LLM during ingestion to place document fragments onto the graph nodes.

The essential point here is that embeddings stored in vector databases are just one form of knowledge base for a retriever to work with. While chunking documents is useful for unstructured prose, we've found it

beneficial to tease out whatever structure we can, and use that structure to support and improve the retriever. Each problem has different ways we can best organize the data for efficient retrieval, and we find it best to use multiple methods to get a worthwhile set of document fragments for later processing.

Query Rewriting

Use an LLM to create several alternative formulations of a query and search with all the alternatives



Anyone who has used search engines knows that it's often best to try different combinations of search terms to find what we're looking for. This is even more apparent with using LLMs, where rephrasing a question often leads to significantly different answers.

We can take advantage of this behavior by getting an LLM to rephrase a query several times, and send each of these queries off for a vector search. We can then combine the results to put in the LLM prompt (often with the help of a [Reranker](#), which we'll discuss shortly).

In our life-sciences example, the user might start with a prompt to explore the tens of thousands of research findings.

Were any of the following clinical findings observed in the study XYZ-1234? Piloerection, ataxia, eyes partially closed, and loose feces?

The rewriter sends this to an LLM, asking it to come up with alternatives.

1. Can you provide details on the clinical symptoms reported in research

XYZ-1234, including any occurrences of goosebumps, lack of coordination, semi-closed eyelids, or diarrhea?

2. In the results of experiment XYZ-1234, were there any recorded observations of hair standing on end, unsteady movement, eyes not fully open, or watery stools?

3. What were the clinical observations noted in trial XYZ-1234, particularly regarding the presence of hair bristling, impaired balance, partially shut eyes, or soft bowel movements?

The optimal number of alternatives varies by dataset: typically, 3-5 variations work best for diverse datasets, while simpler datasets may require up to 3 rewrites. As you tweak query rewrites, use [Evals](#) to track progress.

When to use it

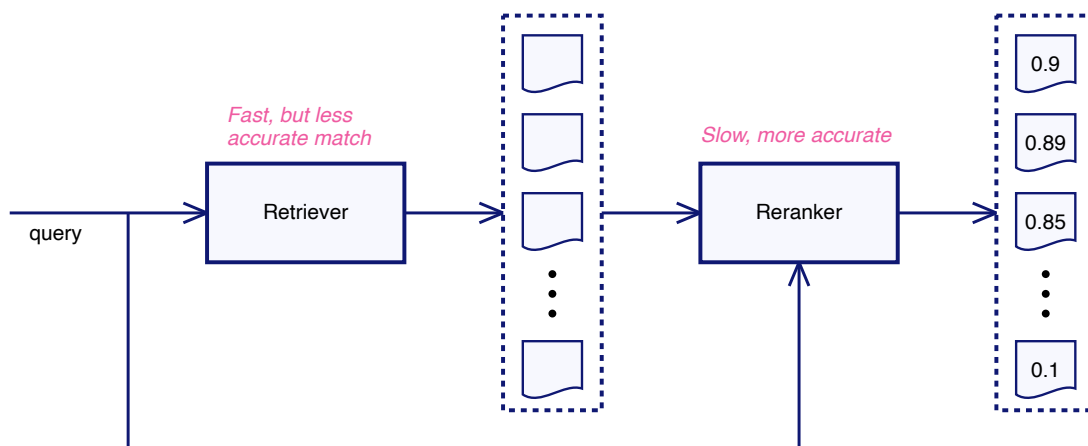
Query rewriting is crucial for complex searches involving multiple subtopics or specialized keywords, particularly in domain-specific vector stores. Creating a few alternative queries can improve the documents that we can find, at the cost of an additional call to an LLM to come up with the alternatives, and additional calls to the retriever to use these alternatives. These additional calls will incur resource costs and increase latency. Teams should experiment to find if the improvement in retrieval is worth these costs.

In our life-sciences engagement, we found it worthwhile to use GPT 4o to create five variations.

Reranker

Rank a set of retrieved document fragments according to their

usefulness and send the best of them to the LLM.



The retriever's job is to find relevant documents quickly, but getting a fast response from the searches leads to lower quality of results. We can try more sophisticated searching, but often complex searches on the whole dataset take too long. In this case we can rapidly generate an overly large set of documents of varying quality and sort them according to how relevant and useful their information is as context for the LLM's prompt.

The reranker can use a deep neural net model, typically a [cross-encoder](#) like [bge-reranker-large](#), to accurately rank the relevance of the input query with the set of retrieved documents. This reranking process is too slow and expensive to do on the entire contents of the vector store, but is worthwhile when it's only considering the candidates returned by a faster, but cruder, search. We can then select the best of these candidates to go into prompt, which stops the prompt from being bloated and the LLM from getting confused by low quality documents.

When to use it

Reranking enhances the accuracy and relevance of the answers in a RAG system. Reranking is worthwhile when there are too many candidates to send in the prompt, or if low quality candidates will reduce

the quality of the LLM's response. Reranking does involve an additional interaction with another AI model, thus adding processing cost and latency to the response, which makes them less suitable for high-traffic applications. Ultimately, choosing to rerank should be based on the specific requirements of a RAG system, balancing the need for high-quality responses with performance and cost limitations.

Another reason to use reranker is to incorporate a user's particular preferences. In the life science chatbot, users can specify preferred or avoided conditions, which are factored into the reranking process to ensure generated responses align with their choices.

We are publishing this article in installments. In future installments we will cover the remaining patterns of a realistic RAG and explore the alternative of Fine Tuning

To find out when we publish the next installment subscribe to this site's [RSS feed](#), or Martin's feeds on [Mastodon](#), [Bluesky](#), [LinkedIn](#), or [X \(Twitter\)](#).
